

# Arquitectura de base de datos V2

LoanOps RD - MySQL oficial, Prisma y NestJS

|                           |                                                        |
|---------------------------|--------------------------------------------------------|
| <b>Documento</b>          | 05 - Arquitectura de base de datos V2                  |
| <b>Version</b>            | 2.0 - MySQL oficial                                    |
| <b>Proyecto</b>           | LoanOps RD                                             |
| <b>Stack oficial</b>      | React + Vite + TypeScript   NestJS + Prisma   MySQL 8+ |
| <b>Autor del proyecto</b> | PholioDev                                              |

Este documento reemplaza la version anterior donde pudiera aparecer PostgreSQL, Neon o Supabase como base ideal. La linea oficial de construccion queda cerrada en MySQL 8+ con NestJS y Prisma.

# Arquitectura de base de datos V2 - MySQL oficial

**Documento:** 05 - Arquitectura de base de datos

**Proyecto:** LoanOps RD - SaaS multiempresa para prestamistas

**Version:** 2.0

**Base de datos oficial:** MySQL 8+

**ORM recomendado:** Prisma

**Backend objetivo:** NestJS + TypeScript

> **Decision oficial de stack:** LoanOps RD se construira con **React + Vite + TypeScript** en frontend, **NestJS + TypeScript** como backend API-first, **Prisma ORM** y **MySQL 8+** como base de datos principal. No se usara PostgreSQL, Neon ni Supabase como base principal en esta fase.

> **Criterio de infraestructura:** se prioriza MySQL porque esta disponible en el entorno actual del proyecto y evita pagar una base de datos adicional. La documentacion queda alineada a esa decision.

## 1. Objetivo del documento

Este documento reemplaza la version anterior de arquitectura de base de datos. El objetivo es dejar una guia tecnica alineada a MySQL 8+, eliminando referencias a PostgreSQL, Neon o Supabase como ruta principal de construccion.

La base de datos debe permitir:

- Separar datos por empresa mediante `empresa_id`.
- Registrar clientes, prestamos, cuotas, pagos, rutas, visitas, promesas y caja.
- Mantener trazabilidad mediante auditoria.
- Soportar planes SaaS, sucursales, usuarios y roles.
- Proteger integridad financiera mediante transacciones.
- Ser compatible con Prisma y NestJS.

## 2. Principios MySQL oficiales

### 2.1 Motor y version

- Usar **MySQL 8+**.
- Usar **InnoDB** como motor obligatorio.
- Usar `utf8mb4` y collation `utf8mb4_unicode_ci` o equivalente.
- Activar claves foraneas reales.

- Usar transacciones en operaciones financieras.

## 2.2 Tipos de datos base

| Caso                   | Tipo MySQL recomendado         | Nota                                                    |
|------------------------|--------------------------------|---------------------------------------------------------|
| ID principal           | BIGINT UNSIGNED AUTO_INCREMENT | Recomendado para MVP y compatibilidad.                  |
| UUID futuro            | CHAR(36)                       | Solo si se decide exponer IDs publicos no secuenciales. |
| Montos                 | DECIMAL(14,2)                  | Nunca usar FLOAT para dinero.                           |
| Porcentajes            | DECIMAL(10,2)                  | Interes, mora y tasas.                                  |
| Fechas de negocio      | DATE                           | Fechas de cuota o ruta.                                 |
| Fechas con hora        | DATETIME                       | Pagos, auditoria, cierres.                              |
| Estados                | ENUM o VARCHAR(40) controlado  | Prisma puede mapear mejor con enums.                    |
| Configuracion flexible | JSON                           | Auditoria, preferencias y payloads.                     |
| Booleanos              | BOOLEAN                        | MySQL lo maneja como TINYINT(1).                        |

## 2.3 Multiempresa primero

Toda entidad operativa debe incluir empresa\_id. Ninguna consulta operativa debe ejecutarse sin scope de empresa, excepto vistas de Super Admin claramente controladas.

Entidades con empresa\_id obligatorio:

- sucursales
- usuarios
- clientes
- prestamos
- cuotas
- pagos
- recibos
- rutas
- ruta\_items
- caja\_movimientos
- promesas\_pago
- visitas\_cliente
- fichas\_cliente
- configuracion\_empresa
- auditoria

- reportes\_exportados

## 2.4 Indices compuestos por empresa

Toda tabla operativa debe tener indices que inicien por empresa\_id cuando se usen para consultas frecuentes.

Ejemplos:

```
CREATE INDEX idx_clientes_empresa_estado ON clientes (empresa_id, estado);
CREATE INDEX idx_cuotas_cobrar_hoy ON cuotas (empresa_id, fecha_programada, estado);
CREATE INDEX idx_pagos_empresa_fecha ON pagos (empresa_id, fecha_pago);
```

## 3. Modelo general de entidades

### 3.1 SaaS y seguridad

| Tabla            | Proposito                                  |
|------------------|--------------------------------------------|
| empresas         | Tenant principal del SaaS.                 |
| sucursales       | Segmentacion operativa por oficina o zona. |
| usuarios         | Acceso al sistema y asignacion operativa.  |
| roles_permisos   | Matriz configurable de permisos.           |
| sesiones_usuario | Control de sesiones activas.               |
| planes_saas      | Planes comerciales del SaaS.               |
| suscripciones    | Relacion empresa-plan.                     |

### 3.2 Operacion crediticia

| Tabla           | Proposito                            |
|-----------------|--------------------------------------|
| clientes        | Clientes o deudores del prestamista. |
| prestamos       | Prestamos otorgados.                 |
| cuotas          | Calendario de pagos.                 |
| pagos           | Cobros completos o parciales.        |
| recibos         | Documentos generados por pagos.      |
| fichas_cliente  | Registro de comportamiento.          |
| promesas_pago   | Compromisos de pago.                 |
| visitas_cliente | Gestiones de campo.                  |

### 3.3 Operacion de campo y control financiero

| Tabla | Proposito       |
|-------|-----------------|
| rutas | Rutas de cobro. |

| Tabla                 | Proposito                                      |
|-----------------------|------------------------------------------------|
| ruta_items            | Clientes/cuotas dentro de ruta.                |
| caja_movimientos      | Entradas, salidas, ajustes y cierres.          |
| cierres_caja          | Cierres diarios o por turno.                   |
| reportes_exportados   | Historial de exportaciones.                    |
| auditoria             | Trazabilidad de acciones criticas.             |
| configuracion_empresa | Reglas de mora, recibos, prestamos y branding. |

## 4. Convenciones de nombres

- Tablas en plural y snake\_case: clientes, caja\_movimientos.
- Columnas en snake\_case: empresa\_id, fecha\_programada.
- Fechas: created\_at, updated\_at, deleted\_at si aplica soft delete.
- Estados: valores en minuscúla: activo, bloqueado, pendiente.
- Montos: prefijo claro: monto\_total, monto\_capital, saldo\_pendiente.

## 5. Tablas principales - MySQL

### 5.1 empresas

```
CREATE TABLE empresas (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre_comercial VARCHAR(180) NOT NULL,
  rnc VARCHAR(30) NULL,
  telefono VARCHAR(30) NULL,
  email VARCHAR(180) NOT NULL,
  direccion TEXT NULL,
  logo_url TEXT NULL,
  plan_id BIGINT UNSIGNED NULL,
  estado ENUM('activa','suspendida','cancelada','prueba') NOT NULL DEFAULT 'prueba',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Reglas:

- Una empresa puede tener varias sucursales.
- Si la empresa esta suspendida, se bloquean nuevas operaciones financieras.
- Super Admin puede ver empresas; usuarios de empresa solo ven la propia.

### 5.2 sucursales

```
CREATE TABLE sucursales (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  empresa_id BIGINT UNSIGNED NOT NULL,
  nombre VARCHAR(140) NOT NULL,
  direccion TEXT NULL,
```

```
telefono VARCHAR(30) NULL,
responsable_id BIGINT UNSIGNED NULL,
estado ENUM('activa','inactiva') NOT NULL DEFAULT 'activa',
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_sucursales_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Indice recomendado:

```
CREATE INDEX idx_sucursales_empresa_estado ON sucursales (empresa_id, estado);
```

## 5.3 usuarios

```
CREATE TABLE usuarios (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
empresa_id BIGINT UNSIGNED NULL,
sucursal_id BIGINT UNSIGNED NULL,
nombre VARCHAR(120) NOT NULL,
apellido VARCHAR(120) NULL,
email VARCHAR(180) NOT NULL,
telefono VARCHAR(30) NULL,
password_hash TEXT NOT NULL,
rol ENUM('SUPER_ADMIN','ADMIN_EMPRESA','SUPERVISOR','COBRADOR') NOT NULL,
estado ENUM('activo','inactivo','suspendido') NOT NULL DEFAULT 'activo',
ultimo_acceso DATETIME NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
UNIQUE KEY uq_usuario_empresa_email (empresa_id, email),
CONSTRAINT fk_usuarios_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE RESTRICT,
CONSTRAINT fk_usuarios_sucursal FOREIGN KEY (sucursal_id) REFERENCES sucursales(id) ON DELETE SET
NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Nota: SUPER\_ADMIN puede tener empresa\_id nulo si opera globalmente.

## 5.4 clientes

```
CREATE TABLE clientes (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
empresa_id BIGINT UNSIGNED NOT NULL,
sucursal_id BIGINT UNSIGNED NULL,
cobrador_id BIGINT UNSIGNED NULL,
nombre_completo VARCHAR(180) NOT NULL,
apodo VARCHAR(80) NULL,
telefono VARCHAR(30) NOT NULL,
cedula VARCHAR(30) NULL,
direccion TEXT NULL,
sector VARCHAR(120) NULL,
referencia_nombre VARCHAR(140) NULL,
referencia_telefono VARCHAR(30) NULL,
estado ENUM('activo','en_riesgo','bloqueado','inactivo') NOT NULL DEFAULT 'activo',
score_actual INT NOT NULL DEFAULT 100,
notas TEXT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_clientes_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE RESTRICT,
CONSTRAINT fk_clientes_sucursal FOREIGN KEY (sucursal_id) REFERENCES sucursales(id) ON DELETE SET
NULL,
CONSTRAINT fk_clientes_cobrador FOREIGN KEY (cobrador_id) REFERENCES usuarios(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Indices:

```
CREATE INDEX idx_clientes_empresa_estado ON clientes (empresa_id, estado);
CREATE INDEX idx_clientes_empresa_cobrador ON clientes (empresa_id, cobrador_id);
CREATE INDEX idx_clientes_empresa_sucursal ON clientes (empresa_id, sucursal_id);
CREATE INDEX idx_clientes_empresa_telefono ON clientes (empresa_id, telefono);
CREATE UNIQUE INDEX uq_clientes_empresa_cedula ON clientes (empresa_id, cedula);
```

## 5.5 prestamos

```
CREATE TABLE prestamos (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  empresa_id BIGINT UNSIGNED NOT NULL,
  cliente_id BIGINT UNSIGNED NOT NULL,
  sucursal_id BIGINT UNSIGNED NULL,
  cobrador_id BIGINT UNSIGNED NULL,
  monto_capital DECIMAL(14,2) NOT NULL,
  interes_tipo ENUM('fijo', 'porcentaje') NOT NULL,
  interes_valor DECIMAL(10,2) NOT NULL,
  frecuencia_pago ENUM('diario', 'semanal', 'quincenal', 'mensual') NOT NULL,
  plazo_cuotas INT NOT NULL,
  fecha_inicio DATE NOT NULL,
  total_pagar DECIMAL(14,2) NOT NULL,
  saldo_pendiente DECIMAL(14,2) NOT NULL,
  cuota_valor DECIMAL(14,2) NOT NULL,
  estado ENUM('activo', 'pagado', 'vencido', 'cancelado', 'refinanciado', 'reprogramado') NOT NULL DEFAULT 'activo',
  observaciones TEXT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_prestamos_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE RESTRICT,
  CONSTRAINT fk_prestamos_cliente FOREIGN KEY (cliente_id) REFERENCES clientes(id) ON DELETE RESTRICT,
  CONSTRAINT fk_prestamos_sucursal FOREIGN KEY (sucursal_id) REFERENCES sucursales(id) ON DELETE SET NULL,
  CONSTRAINT fk_prestamos_cobrador FOREIGN KEY (cobrador_id) REFERENCES usuarios(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Indices:

```
CREATE INDEX idx_prestamos_empresa_cliente_estado ON prestamos (empresa_id, cliente_id, estado);
CREATE INDEX idx_prestamos_empresa_cobrador ON prestamos (empresa_id, cobrador_id);
```

## 5.6 cuotas

```
CREATE TABLE cuotas (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  empresa_id BIGINT UNSIGNED NOT NULL,
  prestamo_id BIGINT UNSIGNED NOT NULL,
  cliente_id BIGINT UNSIGNED NOT NULL,
  numero_cuota INT NOT NULL,
  fecha_programada DATE NOT NULL,
  capital DECIMAL(14,2) NOT NULL DEFAULT 0.00,
  interes DECIMAL(14,2) NOT NULL DEFAULT 0.00,
  mora_generada DECIMAL(14,2) NOT NULL DEFAULT 0.00,
  monto_programado DECIMAL(14,2) NOT NULL,
  monto_pagado DECIMAL(14,2) NOT NULL DEFAULT 0.00,
  saldo_cuota DECIMAL(14,2) NOT NULL,
  estado ENUM('pendiente', 'parcial', 'pagada', 'vencida', 'en_mora') NOT NULL DEFAULT 'pendiente',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uq_cuota_prestamo_numero (prestamo_id, numero_cuota),
  CONSTRAINT fk_cuotas_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE RESTRICT,
  CONSTRAINT fk_cuotas_prestamo FOREIGN KEY (prestamo_id) REFERENCES prestamos(id) ON DELETE RESTRICT,
  CONSTRAINT fk_cuotas_cliente FOREIGN KEY (cliente_id) REFERENCES clientes(id) ON DELETE RESTRICT
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Indice Cobrar Hoy:

```
CREATE INDEX idx_cuotas_cobrar_hoy ON cuotas (empresa_id, fecha_programada, estado);
```

## 5.7 pagos

```
CREATE TABLE pagos (  
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  empresa_id BIGINT UNSIGNED NOT NULL,  
  cliente_id BIGINT UNSIGNED NOT NULL,  
  prestamo_id BIGINT UNSIGNED NOT NULL,  
  cuota_id BIGINT UNSIGNED NULL,  
  monto_capital DECIMAL(14,2) NOT NULL DEFAULT 0.00,  
  monto_interes DECIMAL(14,2) NOT NULL DEFAULT 0.00,  
  monto_mora DECIMAL(14,2) NOT NULL DEFAULT 0.00,  
  monto_total DECIMAL(14,2) NOT NULL,  
  metodo_pago ENUM('efectivo','transferencia','tarjeta','otro') NOT NULL DEFAULT 'efectivo',  
  registrado_por BIGINT UNSIGNED NOT NULL,  
  fecha_pago DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  estado ENUM('aplicado','anulado') NOT NULL DEFAULT 'aplicado',  
  observacion TEXT NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_pagos_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE RESTRICT,  
  CONSTRAINT fk_pagos_cliente FOREIGN KEY (cliente_id) REFERENCES clientes(id) ON DELETE RESTRICT,  
  CONSTRAINT fk_pagos_prestamo FOREIGN KEY (prestamo_id) REFERENCES prestamos(id) ON DELETE RESTRICT,  
  CONSTRAINT fk_pagos_cuota FOREIGN KEY (cuota_id) REFERENCES cuotas(id) ON DELETE SET NULL,  
  CONSTRAINT fk_pagos_usuario FOREIGN KEY (registrado_por) REFERENCES usuarios(id) ON DELETE RESTRICT  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

## 5.8 recibos

```
CREATE TABLE recibos (  
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  empresa_id BIGINT UNSIGNED NOT NULL,  
  pago_id BIGINT UNSIGNED NOT NULL,  
  cliente_id BIGINT UNSIGNED NOT NULL,  
  prestamo_id BIGINT UNSIGNED NOT NULL,  
  numero_recibo VARCHAR(60) NOT NULL,  
  pdf_url TEXT NULL,  
  estado ENUM('generado','enviado','anulado') NOT NULL DEFAULT 'generado',  
  enviado_por ENUM('whatsapp','correo','descarga') NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE KEY uq_recibo_empresa_numero (empresa_id, numero_recibo),  
  CONSTRAINT fk_recibos_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE RESTRICT,  
  CONSTRAINT fk_recibos_pago FOREIGN KEY (pago_id) REFERENCES pagos(id) ON DELETE RESTRICT  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

## 5.9 caja\_movimientos

```
CREATE TABLE caja_movimientos (  
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  empresa_id BIGINT UNSIGNED NOT NULL,  
  sucursal_id BIGINT UNSIGNED NULL,  
  tipo_movimiento ENUM('entrada','salida','ajuste','cierre') NOT NULL,  
  origen ENUM('pago','manual','cierre','reverso') NOT NULL,  
  pago_id BIGINT UNSIGNED NULL,  
  monto DECIMAL(14,2) NOT NULL,  
  concepto VARCHAR(180) NOT NULL,  
  registrado_por BIGINT UNSIGNED NOT NULL,  
  fecha_movimiento DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
estado ENUM('aplicado','anulado') NOT NULL DEFAULT 'aplicado',
CONSTRAINT fk_caja_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE RESTRICT,
CONSTRAINT fk_caja_sucursal FOREIGN KEY (sucursal_id) REFERENCES sucursales(id) ON DELETE SET NULL,
CONSTRAINT fk_caja_pago FOREIGN KEY (pago_id) REFERENCES pagos(id) ON DELETE SET NULL,
CONSTRAINT fk_caja_usuario FOREIGN KEY (registrado_por) REFERENCES usuarios(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

## 5.10 auditoria

```
CREATE TABLE auditoria (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
empresa_id BIGINT UNSIGNED NULL,
usuario_id BIGINT UNSIGNED NULL,
accion VARCHAR(120) NOT NULL,
entidad VARCHAR(120) NOT NULL,
entidad_id BIGINT UNSIGNED NULL,
detalle_json JSON NULL,
ip VARCHAR(80) NULL,
user_agent TEXT NULL,
fecha_evento DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_auditoria_empresa FOREIGN KEY (empresa_id) REFERENCES empresas(id) ON DELETE SET NULL,
CONSTRAINT fk_auditoria_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Indices:

```
CREATE INDEX idx_auditoria_empresa_fecha ON auditoria (empresa_id, fecha_evento);
CREATE INDEX idx_auditoria_empresa_usuario ON auditoria (empresa_id, usuario_id);
```

## 6. Reglas transaccionales criticas

### 6.1 Crear prestamo

Debe ejecutarse dentro de una transaccion Prisma:

1. Validar empresa activa.
2. Validar cliente activo y no bloqueado.
3. Calcular total y calendario en backend.
4. Crear prestamo.
5. Crear cuotas.
6. Registrar auditoria.
7. Devolver detalle del prestamo.

Si falla la generacion de cuotas, el prestamo no debe quedar creado.

### 6.2 Registrar pago

Debe ser una transaccion atomica:

1. Validar usuario, empresa, cliente, prestamo y cuota.

2. Validar que el monto no exceda el saldo permitido.
3. Crear registro de pago.
4. Actualizar cuota o cuotas afectadas.
5. Actualizar saldo del prestamo.
6. Crear movimiento de caja.
7. Crear recibo.
8. Registrar auditoria.

Si falla caja o auditoria, se revierte el pago completo.

### 6.3 Anular pago

No se debe borrar el pago. Debe cambiar a anulado y crear movimiento compensatorio.

Pasos:

1. Validar permiso.
2. Cambiar pago a anulado.
3. Revertir montos de cuota/prestamo.
4. Crear movimiento de caja tipo reverso.
5. Anular recibo.
6. Registrar auditoria con motivo.

### 6.4 Cerrar caja

1. Calcular monto teorico.
2. Recibir monto real.
3. Calcular diferencia.
4. Crear movimiento/cierre.
5. Bloquear periodo cerrado.
6. Registrar auditoria.

## 7. Prisma schema base recomendado

```
datasource db {
  provider = "mysql"
  url = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model Empresa {
  id BigInt @id @default(autoincrement()) @db.UnsignedBigInt
```

```

nombreComercial String @map("nombre_comercial") @db.VarChar(180)
rnc String? @db.VarChar(30)
telefono String? @db.VarChar(30)
email String @db.VarChar(180)
direccion String? @db.Text
logoUrl String? @map("logo_url") @db.Text
estado String @default("prueba") @db.VarChar(30)
createdAt DateTime @default(now()) @map("created_at")
updatedAt DateTime @updatedAt @map("updated_at")

sucursales Sucursal[]
usuarios Usuario[]
clientes Cliente[]

@@map("empresas")
}

model Cliente {
  id BigInt @id @default(autoincrement()) @db.UnsignedBigInt
  empresaId BigInt @map("empresa_id") @db.UnsignedBigInt
  sucursalId BigInt? @map("sucursal_id") @db.UnsignedBigInt
  cobradorId BigInt? @map("cobrador_id") @db.UnsignedBigInt
  nombreCompleto String @map("nombre_completo") @db.VarChar(180)
  telefono String @db.VarChar(30)
  cedula String? @db.VarChar(30)
  estado String @default("activo") @db.VarChar(30)
  scoreActual Int @default(100) @map("score_actual")
  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @updatedAt @map("updated_at")

  empresa Empresa @relation(fields: [empresaId], references: [id])

  @@index([empresaId, estado])
  @@index([empresaId, cobradorId])
  @@unique([empresaId, cedula])
  @@map("clientes")
}

```

## 8. Reglas para consultas backend

Toda consulta operativa debe aplicar scope:

```

const empresaId = currentUser.empresaId;

return this.prisma.cliente.findMany({
  where: {
    empresaId,
    estado: 'activo',
  },
});

```

Excepcion:

- Super Admin solo en modulos globales.
- Modo soporte debe ser auditado.

## 9. Backup, migraciones y mantenimiento

## 9.1 Migraciones

Usar Prisma Migrate:

```
npx prisma migrate dev  
npx prisma migrate deploy
```

## 9.2 Backups

- Backup diario automatico de MySQL.
- Backup antes de migraciones.
- Retencion minima recomendada: 14 a 30 dias.
- Prueba mensual de restauracion.

## 9.3 Seguridad

- Usuario MySQL con permisos limitados.
- No usar usuario root desde la aplicacion.
- Variables en .env.
- Rotacion de credenciales si hay filtracion.

## 10. Checklist QA de base de datos

- Todas las tablas operativas tienen empresa\_id.
- Pagos se registran en transaccion.
- Pago afecta cuota, prestamo, caja, recibo y auditoria.
- Cliente bloqueado no puede recibir prestamo.
- Ruta cerrada no permite edicion libre.
- Caja cerrada no permite movimientos no autorizados.
- Indices por empresa\_id creados.
- Montos usan DECIMAL, nunca FLOAT.
- Auditoria usa campo JSON compatible con MySQL.
- Prisma usa provider = "mysql".
- No quedan referencias operativas a PostgreSQL, Neon o Supabase.

## 11. Roadmap tecnico de datos

### MVP

- Empresas.

- Sucursales.
- Usuarios.
- Clientes.
- Prestamos.
- Cuotas.
- Pagos.
- Recibos.
- Caja basica.
- Auditoria basica.

## Pro

- Rutas.
- Promesas.
- Visitas.
- Fichas.
- Scoring.
- Reportes exportados.
- Configuracion avanzada.

## Escala

- Planes SaaS.
- Suscripciones.
- Facturacion.
- Jobs automaticos.
- App movil offline.
- Auditoria avanzada.
- Analitica e IA.

## 12. Conclusion

La arquitectura oficial de base de datos de LoanOps RD queda basada en **MySQL 8+**, usando InnoDB, Prisma y transacciones. Esta decision reduce costos de infraestructura, se adapta al servidor actual y permite construir el SaaS sin depender de PostgreSQL, Neon o Supabase.

La prioridad tecnica es proteger la integridad financiera: ningun pago debe existir sin reflejarse en cuota, prestamo, caja, recibo y auditoria.